

StackForge — Informe Final (Stage III)

Contents

1. Introducción	1
2. Modelo Computacional	2
2.1. Dominio	2
2.2. Lenguaje	2
3. Implementación	4
3.1. Frontend	4
3.2. Backend	5
3.4. Dificultades Encontradas	6
4. Futuras Extensiones	7
5. Conclusiones	7
6. Referencias	7
7. Bibliografía	8

Nombre	Apellido	Legajo	E-mail
Joaquín	Nolasco de Carlés	65368	jnolascodecarles@itba.edu.ar
Nicanor	Novotny	65530	nnovotny@itba.edu.ar
Agustín	Korman	65116	agkorman@itba.edu.ar

1. Introducción

StackForge es un lenguaje de dominio específico (DSL) para describir la topología de una aplicación multicontenedor, junto con el compilador que lo traduce a artefactos de Docker Compose. Está construido en C sobre Flex y Bison, partiendo del proyecto base Flex-Bison-Compiler v2.0.0 provisto por la cátedra.

La idea de fondo no cambió respecto del Stage I: describir aplicaciones de contenedores mediante herramientas de despliegue tradicionales obliga a escribir a mano una cantidad de detalle estructural que, las más de las veces, se deduce del rol que cumple cada componente y de cómo se relaciona con

los demás. StackForge sube el nivel de abstracción. El usuario declara qué servicios hay, qué rol topológico tiene cada uno y cómo se conectan; el compilador deriva las redes, las dependencias, los puertos publicados y los volúmenes, y escribe un `docker-compose.yml` consistente con esa descripción.

Lo que sí cambió, y bastante, es el lenguaje concreto. La propuesta original giraba alrededor de roles de capa (`frontend`, `api`, `database`, `cache`) y una palabra clave `connects_to` para la conectividad. Durante la implementación reorientamos el modelo hacia la topología de red: aparecieron las redes explícitas (`network`), los roles pasaron a describir la posición de un servicio en esa topología (`proxy`, `service`, `static`, `database`, `cache`), la conectividad se expresa con un operador `->`, y se sumaron volúmenes y montajes. El porqué de ese giro, y lo que costó, se cuenta en las secciones 2 y 3.4.

Este informe describe el desarrollo tal como quedó en la rama `development` del repositorio: el dominio y el lenguaje (sección 2), la implementación de las dos mitades del compilador (sección 3), las dificultades que encontramos (sección 3.4), lo que quedó propuesto pero sin implementar (sección 4) y un cierre con conclusiones.

2. Modelo Computacional

2.1. Dominio

El dominio es la especificación declarativa de arquitecturas multicontenedor para entornos de desarrollo y prueba. Lo que le importa al usuario no es cada línea del archivo de despliegue, sino los componentes lógicos de la aplicación y las relaciones entre ellos.

El concepto central es el de servicio caracterizado por un rol. A diferencia del Stage I, donde el rol nombraba la capa funcional, en la versión implementada el rol describe la posición topológica del servicio y, con ella, qué puede y qué no puede hacer:

- **proxy**: punto de entrada público, pensado para exponerse al host y para enrutar tráfico hacia el resto de la aplicación (típicamente un reverse-proxy como nginx).
- **service**: servicio de aplicación genérico, interno, que implementa lógica de negocio.
- **static**: servidor de contenido estático.
- **database**: almacenamiento persistente. Interno por diseño: no se puede exponer.
- **cache**: almacenamiento temporal o de sesión. También interno por diseño.

Sobre esos servicios el dominio define un puñado de relaciones: la exposición de un servicio hacia el host a través de un puerto, la conectividad dirigida entre dos servicios (uno necesita hablar con otro), la pertenencia de los servicios a redes, los enlaces entre redes, y la persistencia mediante volúmenes y montajes. La distinción entre servicios públicos e internos no es decorativa: que una `database` no pueda exponerse, o que dos redes no se comuniquen salvo que exista un enlace explícito entre ellas, son reglas del dominio que el compilador hace cumplir.

La decisión de modelar la topología de red de forma explícita, en lugar de inferir todo a partir del rol, es la diferencia conceptual más importante con el Stage I. Permite describir desde una aplicación de tres capas en una sola red hasta una plataforma de microservicios con varias redes interconectadas, distinguiendo qué es público y qué es interno, sin abandonar la sintaxis compacta.

2.2. Lenguaje

Un programa StackForge declara una aplicación con un nombre y un cuerpo. El cuerpo se compone de bloques `network` y, opcionalmente, de enlaces entre redes. Dentro de cada red se declaran los servicios y todo lo que les concierne: exposiciones, conexiones, volúmenes y montajes.

La forma más simple es una única red con un único servicio:

```
app SingleService {
  network main {
    service backend using "node:18";
  }
}
```

Código 1: aplicación mínima con un solo servicio interno.

Cada servicio se declara con la forma `<rol> <nombre> using "<imagen>";`. El nombre es un identificador y la imagen es una cadena literal (la referencia de imagen de Docker, etiqueta incluida). Sobre esa base se agregan las relaciones. La exposición usa `expose <servicio> on <puerto>;`, y la conectividad, el operador `->`:

```
app ThreeTier {
  network main {
    proxy web using "nginx:latest";
    service backend using "node:20";
    database db using "postgres:16";
    expose web on 80;
    web -> backend;
    backend -> db;
  }
}
```

Código 2: aplicación de tres capas en una sola red.

El operador `->` reemplazó a la palabra clave `connects_to` de la propuesta original. La flecha es direccional y se lee igual que en un diagrama de arquitectura: `web -> backend` significa que `web` necesita alcanzar a `backend`. De esa relación el compilador deriva tanto la dependencia (`depends_on`) como la pertenencia de red necesaria para que la comunicación sea posible.

Cuando una aplicación tiene varias redes, los servicios de cada red quedan aislados salvo que se declare un enlace entre las redes al nivel de la aplicación, con la misma flecha. Esto habilita topologías de microservicios donde lo público y lo interno están separados:

```
app Platform {
  network public {
    proxy lb using "nginx:1.25";
    expose lb on 443;
  }
  network users {
    service users_api using "node:20";
    database users_db using "postgres:16";
    users_api -> users_db;
    volume users_data;
    mount users_data on users_db at "/var/lib/postgresql/data";
  }
  network billing {
    service billing_api using "python:3.12";
    cache queue using "redis:7";
    billing_api -> queue;
  }
}
```

```

    mount "./billing" on billing_api at "/app";
}
public -> users;
public -> billing;
users -> billing;
}

```

Código 3: plataforma de microservicios con redes interconectadas, volúmenes y montajes.

El Código 3 muestra las dos construcciones que se sumaron respecto del Stage I. Los volúmenes se declaran con `volume <nombre>;` y se montan con `mount <volumen> on <servicio> at "<ruta>";`, para persistir datos de una base. La misma palabra clave `mount` admite, en lugar de un volumen nombrado, una ruta del host entre comillas: `mount "./billing" on billing_api at "/app";`. Esa variante genera un bind mount, pensada para montar código local dentro del contenedor durante el desarrollo. La gramática distingue ambos casos por el tipo del primer operando (identificador contra cadena), y el backend los traduce a entradas de Compose distintas.

La gramática es deliberadamente plana. Una aplicación es una secuencia de items (redes o enlaces de red); una red es una secuencia de declaraciones; y cada declaración es una de seis formas: servicio, exposición, conexión, volumen, montaje-de-volumen o montaje-de-host. No hay anidamiento más allá de `app > network > declaración`, lo que mantiene el lenguaje fácil de leer y la gramática libre de ambigüedades.

Los comentarios de línea se escriben con `//` y el lexer los descarta. Las palabras clave (`app`, `network`, los cinco roles, `using`, `expose`, `on`, `volume`, `mount`, `at`) son reservadas: un nombre como `frontend` —que en el Stage I era un rol— hoy se lexa como un simple identificador, no como palabra clave, y por lo tanto no sirve como rol.

3. Implementación

El compilador conserva la estructura del proyecto base: un frontend con análisis léxico y sintáctico que construye un AST, y un backend con análisis semántico y generación de código. El punto de entrada (`EntryPoint.c`) encadena las fases —análisis sintáctico, semántico y generación— y aborta en la primera que falle. El estado compartido entre fases (el AST y la tabla de símbolos) viaja en una estructura `CompilerState`.

3.1. Frontend

El análisis léxico está definido en `FlexPatterns.l`. Cada patrón delega en una acción nombrada de `FlexActions.c` que sigue el patrón del proyecto base: crear el token, poblar su valor semántico si corresponde, registrarlo, empujarlo al parser y destruirlo. El orden de las reglas importa: las palabras clave se listan antes que el patrón de identificador, de modo que `service` matchea como token `SERVICE` y no como `ID`. Las cadenas se reconocen con comillas pero su valor semántico se almacena sin ellas, porque lo que interesa aguas abajo es el contenido (la referencia de imagen, la ruta de montaje). Los espacios y los comentarios de línea se ignoran; cualquier carácter que no encaje en ningún patrón produce un token `UNKNOWN` y corta la compilación con error.

Los literales enteros tienen un cuidado extra. Como un puerto se valida después contra el rango 1–65535, conviene que un literal fuera del rango de `int` no se vuelva, por desbordamiento, un número que parezca válido. Por eso la acción del entero usa `strtol` con control de `errno` y, ante un `ERANGE` o

un valor mayor que `INT_MAX`, fuerza el valor a `-1`. Un `-1` nunca pasa la validación de puerto, así que un literal monstruoso se rechaza limpiamente en lugar de colarse envuelto.

El análisis sintáctico (`BisonGrammar.y`) usa el parser push y puro heredado del proyecto base, con `%locations` y la unión de valores semánticos. La gramática es la descrita en 2.2; cada producción invoca una acción semántica de `BisonActions.c` que construye el nodo de AST correspondiente. Las listas (items de la aplicación, declaraciones de una red) se arman como listas simplemente enlazadas que se recorren hasta el final para anexas, conservando el orden de aparición —un detalle que después aprovecha el generador para emitir las cosas en el mismo orden en que el usuario las escribió.

El AST (`AbstractSyntaxTree.h`) refleja el dominio sin adornos: una `App` con su nombre y su lista de `AppItem`, donde cada item es una red o un enlace de red; una `Network` con su nombre y su lista de `Declaration`; y una `Declaration` que, según su tipo, apunta a una de las cinco estructuras concretas (servicio, exposición, conexión, volumen, montaje) mediante una unión. Se definieron destructores para cada nodo, registrados además como destructores de Bison para que un parseo fallido no deje memoria colgada.

3.2. Backend

El backend tiene dos responsabilidades: validar que el programa tenga sentido y, si lo tiene, generar los artefactos.

El análisis semántico (`SemanticAnalyzer.c`) trabaja en dos pasadas sobre el AST, apoyado en una tabla de símbolos (`SymbolTable.c/.h`). La primera pasada declara cada red, servicio y volumen, rechazando duplicados. Acá hay una decisión que tuvo consecuencias: los nombres de servicio y de volumen son globales, porque terminan siendo claves del archivo Compose, que no admite dos servicios con el mismo nombre aunque vivan en redes distintas. La tabla, entonces, guarda para cada símbolo su nombre, su red y su rol, y ofrece tanto una búsqueda con alcance de red como una búsqueda global; los duplicados se detectan en ambos niveles. Los símbolos no son dueños de sus cadenas: aliasan la memoria del AST, lo que obliga a destruir la tabla antes que el AST (ver 3.4).

La segunda pasada valida referencias y reglas del dominio. Las verificaciones que implementamos son varias: que la aplicación declare al menos un servicio y que toda imagen sea no vacía; que un `expose` apunte a un servicio existente, con un puerto en el rango válido, y que no se exponga una `database` ni un `cache`; que no haya dos exposiciones iguales ni dos servicios publicando el mismo puerto del host; que toda conexión $\rightarrow$ referencie servicios existentes, no sea reflexiva, y esté declarada en la red del servicio origen; que una conexión entre redes distintas solo se permita si existe el enlace de red a nivel de aplicación; que un enlace de red referencie redes declaradas; y que un montaje apunte a un servicio existente y, si es de volumen, a un volumen declarado en la misma red. Cada violación se acumula en un contador de errores y se reporta; si al terminar la pasada hubo al menos un error, el programa se rechaza.

La generación de código (`Generator.c`) recorre el AST ya validado y escribe, bajo `output/<app>/`, un `docker-compose.yml` más un andamiaje por servicio. El archivo Compose se arma en tres bloques. Para cada servicio emite su imagen, las redes a las que pertenece, los puertos publicados (a partir de los `expose`), las dependencias (`depends_on`, a partir de las flechas que salen del servicio) y los volúmenes montados —distinguiendo el volumen nombrado del bind mount, que se emite con la forma larga `type: bind`. Después emite el mapa de redes y, si hubo volúmenes, el mapa de volúmenes.

El cálculo de a qué redes pertenece un servicio es la parte menos obvia del generador. Un servicio siempre está en su propia red; además, si tiene una flecha hacia un servicio de otra red, se lo agrega a la red del destino para que la conexión sea efectivamente alcanzable; y un `proxy` se suma a las redes hacia

las que apunta su red mediante enlaces de nivel de aplicación, que es lo que le permite hacer de frente público de varios backends. Las rutas de los bind mounts se emiten tal cual las escribió el usuario, y como Docker Compose resuelve las rutas relativas respecto del directorio del archivo generado, una ruta como `./src` queda relativa a `output/<app>/` y no al directorio donde corrió el compilador; es una sutileza que conviene tener presente y que está documentada.

El andamiaje complementa al Compose: por cada servicio se crea una carpeta `output/<app>/services/<nombre>/` con un `Dockerfile` stub que parte de la imagen del servicio y agrega comentarios orientados al rol (un `proxy` trae recordatorios para copiar su configuración de reverse-proxy, un `static` para copiar sus assets, un `service` un `WORKDIR` y los pasos típicos de empaquetado). Es un punto de partida, no una configuración funcional: los stubs guían, pero todavía no escriben, por ejemplo, un `nginx.conf` real cableado a los upstreams. Esa parte quedó como extensión futura (sección 4).

La validez del artefacto no se da por sentada: el script de pruebas corre `docker compose config` sobre cada `docker-compose.yml` generado, de modo que un caso de aceptación no solo tiene que compilar, sino producir un Compose que el propio Docker considere válido.

El conjunto de pruebas terminó con 27 casos de aceptación y 42 de rechazo en `src/test/c`. Los de aceptación cubren desde el servicio único hasta la plataforma multired completa, pasando por cada rol, exposiciones, conexiones, volúmenes y montajes de host (incluyendo rutas con espacios y rutas relativas). Los de rechazo cubren tanto errores sintácticos (bloques incompletos, delimitadores ausentes, el viejo `connects_to`, un rol inexistente) como cada una de las reglas semánticas: nombres duplicados, puertos inválidos o en conflicto, exposición de servicios internos, conexiones reflexivas o entre redes sin enlace, montajes a símbolos inexistentes, y demás.

3.4. Dificultades Encontradas

La dificultad más grande fue de diseño, no de código: darnos cuenta, ya con la propuesta del Stage I en la mano, de que modelar el dominio por capas funcionales (`frontend`, `api`) y dejar las redes implícitas se quedaba corto. Las arquitecturas que queríamos poder expresar —varios servicios del mismo tipo, redes separadas, un proxy público frente a backends internos— pedían que la red fuera una construcción de primera clase. Rehacer el lenguaje alrededor de la topología implicó cambiar los roles, reemplazar `connects_to` por `->` y reescribir buena parte de la gramática y del AST. Fue la decisión correcta, pero costó una iteración entera.

Una vez con redes explícitas, la conectividad entre redes trajo su propia cola de casos límite. Decidir que una conexión `->` debe declararse en la red del servicio origen, y que cruzar de una red a otra exige un enlace explícito a nivel de aplicación, son reglas que parecen naturales pero que se llenan de excepciones: el origen en otra red, el destino inexistente, la conexión reflexiva, el enlace de red hacia una red no declarada. Varias rondas de commits (“more edge cases”, “now yes, last edge cases”) fueron justamente eso: encontrar y cerrar esos huecos uno por uno, cada uno con su caso de rechazo.

La gestión de memoria entre el AST y la tabla de símbolos fue otra fuente de cuidado. Como los símbolos aliasan las cadenas del AST en lugar de copiarlas, el orden de destrucción dejó de ser indiferente: hay que liberar la tabla antes que el programa, o se liberan dos veces las mismas cadenas. Quedó documentado en el header de la tabla y respetado en el punto de entrada, pero es la clase de detalle que muerde si uno lo olvida.

Hubo además sutilezas puntuales que llevaron más tiempo del esperado. El desbordamiento de literales enteros, que resolvimos mapeando todo lo fuera de rango a `-1` para que no se colara ningún puerto envuelto. La resolución de rutas relativas en los bind mounts, que Compose hace respecto del archivo generado y no del directorio de trabajo, lo que obligó a documentar el comportamiento para no

sorprender al usuario. Y el hecho de que los nombres de servicio y volumen tengan que ser globales aunque se declaren dentro de una red, porque son claves de Compose: eso forzó la búsqueda global en la tabla y una segunda capa de detección de duplicados.

4. Futuras Extensiones

Buena parte de las mejoras que la cátedra sugirió en la devolución del Stage I terminaron implementadas en esta entrega: la generación de carpetas y Dockerfiles por servicio, la sintaxis de conexión con `->` en lugar de `connects_to`, los volúmenes y su montaje contra el host, y una topología de red más rica con redes interconectadas y la distinción entre servicios públicos e internos. Quedan, sin embargo, frentes abiertos.

El más concreto es completar el andamiaje de configuración para que sea funcional y no solo declarativo. Hoy el generador deja un `Dockerfile` stub con comentarios orientados al rol, pero no escribe la configuración real: un proxy nginx no recibe todavía un `nginx.conf` generado que lo cablee como reverse-proxy hacia los servicios a los que apunta con `->`, ni un `static` obtiene una configuración mínima de servidor de contenido estático. El compilador tiene toda la información para hacerlo —conoce las flechas, los roles y las imágenes—, de modo que la extensión natural es derivar esos archivos de configuración a partir de la topología ya validada.

Sobre esa base quedan extensiones que el modelo actual no contempla y que serían el siguiente paso lógico:

- Inyección de configuración para servicios internos: variables de entorno y secretos para bases de datos y caches (credenciales, nombres de base), que hoy el usuario debe completar a mano sobre el Compose generado.
- Políticas operativas como `restart` y `healthcheck` derivadas del rol, para acercar el artefacto a algo desplegable sin retoques.

5. Conclusiones

El proyecto cumplió el objetivo: a partir de una especificación declarativa y compacta, el compilador produce un `docker-compose.yml` válido —verificado contra el propio Docker— más un andamiaje por servicio. El recorrido completo, de léxico a generación, quedó cubierto por un conjunto de pruebas amplio que ejercita tanto los caminos felices como cada regla de rechazo.

La lección que nos llevamos es sobre el valor de revisar el diseño del lenguaje antes de apurarse a implementarlo. La propuesta del Stage I era razonable, pero recién al intentar expresar arquitecturas concretas se hizo evidente que la red tenía que ser explícita. Haber pagado el costo de rehacer el modelo temprano nos dio un lenguaje que escala desde un servicio suelto hasta una plataforma multired, sin volverse verboso. El resto del trabajo —las validaciones semánticas, los casos límite de conectividad entre redes, las sutilezas de memoria y de rutas— fue, en buena medida, consecuencia de haber tomado esa decisión y haberla sostenido hasta sus últimas implicancias.

6. Referencias

Material citado explícitamente en el informe.

- Docker, Inc. *Compose file reference* (servicios, redes, `depends_on`, `ports`, volúmenes y bind mounts). <https://docs.docker.com/compose/compose-file/>

- Cátedra de Autómatas, Teoría de Lenguajes y Compiladores (ITBA). *StackForge* — *Especificación (Stage I)*, doc/STAGE 1.pdf.
- Golmar, A. Proyecto base *Flex-Bison-Compiler* (v2.0.0). <https://github.com/agustingolmar/Flex-Bison-Compiler>
- StackForge — repositorio de la solución del grupo. <https://github.com/agkorman/Flex-Bison-Compiler>

7. Bibliografía

Material consultado pero no citado explícitamente.

- The Flex Project. *Lexical Analysis with Flex* (manual de Flex). <https://westes.github.io/flex/manual/>
- Free Software Foundation. *Bison Manual* (parser push, parser puro, destructores, ubicaciones). <https://www.gnu.org/software/bison/manual/>
- Docker, Inc. *Docker documentation* (CLI, networking y volumes). <https://docs.docker.com/>
- Cátedra de Autómatas, Teoría de Lenguajes y Compiladores (ITBA). Apuntes y teóricas sobre análisis léxico, sintáctico y construcción de compiladores.
- *YAML Ain't Markup Language (YAML) Specification*, versión 1.2.2. <https://yaml.org/spec/1.2.2/>